

# 12-WEEK DAILY BUILD PLAN

## AI-Augmented Security Posture Intelligence Platform

*Sathish & Karthik · Skill-aligned task split · Customer-ready MVP · Next.js 15 frontend*

### Why this split is different from the original plan

Original plan assigned Karthik the FastAPI/Celery/Python backend, but Karthik's 15+ years are entirely Angular/React/TypeScript frontend (Citi, Buckman, PushPro, Auriss). Sathish has Python, C++, ML, cryptography, network security (IDS/botnet at Auriss), cloud SDKs, PKI, and post-quantum TLS work at ICU Medical. The realistic split:

- Sathish owns: Data Store, Backend skeleton, Tool adapters, Threat-intel + AI layer, Security review.
- Karthik owns: Entire frontend (Next.js 15 + shadcn/ui — leverages his React/TypeScript fluency), Docker/Compose builds, CI/CD, production deploy.
- Both pair on: Week 1 setup, weekly integration check, final demo.

### Frontend stack rationale (Next.js 15 + adapted dependencies)

The originally proposed Next.js 13 + NextAuth + bcryptjs + react-icons stack was adapted to avoid common pitfalls in production builds dated May 2026:

- Next.js 15 (not 13): App Router stabilized in 14/15; React 18 server components; better caching and middleware semantics. No migration tax 6 months in.
- shadcn/ui added: Tailwind alone provides utility classes, not components. shadcn gives Radix-based, accessible, copy-paste-into-your-repo components — Dialog, DataTable, Form, Sheet, Toast — without the dependency lock-in of MUI.
- Auth.js v5 (NextAuth's current name) Credentials provider: proxies authentication to FastAPI /auth/token. FastAPI remains source of truth for JWT and password hashing.
- bcryptjs removed from frontend: Password hashing belongs on the backend (passlib[bcrypt] in FastAPI). Hashing client-side leaks no information and adds attack surface.
- lucide-react (not react-icons): Same Feather glyphs, single-purpose package, better tree-shaking, ~70% smaller bundle when only icons used.
- Inter via next/font/google: Self-hosted at build time, zero CLS, no third-party request.
- react-dropzone retained: Mature, well-maintained, no concerns.

## MVP scope reduction (so 12 weeks is realistic)

The original 6-scanner plan is too much for a first paying customer in 12 weeks. Cut to v1.1:

- KEEP for MVP: Nuclei (vuln) · Trivy (SCA/container) · Gitleaks (secrets) · NVD · CISA KEV · EPSS · Claude API · Jira · Slack · PDF report
- DEFER to v1.1: Prowler (cloud) · Checkov (IaC) · Nmap · MITRE kill-chain clusters · OWASP heatmap · compliance reports · recurring scans

Rationale: 3 scanners (web app + container + repo) cover ~80% of mid-market buyer asks. Ship narrow, expand after revenue.

## Differentiators vs DefectDojo (the closest competitor)

DefectDojo is an ingest/aggregation platform — they take findings from 200+ scanners and process them. They cannot run scans or hold source code at scan time. That architectural fact creates features they structurally cannot deliver. v1.0 includes six of them, each baked into the daily plan:

- FR-SCN-12 — PR-time scanning via GitHub App (Week 11, Days 53–54): scan the PR diff before merge, post inline file:line annotations + checks. DefectDojo ingests after-the-fact and cannot block merges.
- FR-SCN-13 — Exploit validation mode (Week 4, Day 18): Nuclei -validate confirms exploitability for critical findings. DefectDojo only has the finding, not the scanner — cannot validate.
- FR-AI-08 — Source-call reachability (Week 6, Day 28): parse the cloned repo's AST to check whether vulnerable functions are actually called from your code, not just imported transitively. DefectDojo lacks source access.
- FR-AI-10 — Agentic chat over scan data (Week 6, Day 29): Claude with tool-use answers 'what should I fix this sprint?' over your live data. DefectDojo exposes data via MCP but does not ship a native agent.
- FR-OUT-04 — AI-generated fix PRs (Week 7, Day 33): open actual git PRs for Gitleaks + Trivy SCA fixes. DefectDojo creates Jira tickets — we create commits.
- FR-UI-08 — Single-screen CISO view (Week 8, Day 39): one URL designed for projection. DefectDojo is configurable dashboards; we are opinionated.

## Stack at a glance

| Layer          | Technology                                         | Owner   |
|----------------|----------------------------------------------------|---------|
| Backend API    | FastAPI + Uvicorn + Pydantic v2                    | Sathish |
| Database       | PostgreSQL 16 (JSONB for raw tool output)          | Sathish |
| Cache / Queue  | Redis 7 + Celery 5                                 | Sathish |
| Object Storage | MinIO (PDF reports, repo clones)                   | Sathish |
| Backend Auth   | FastAPI JWT (python-jose) + passlib[bcrypt] + RBAC | Sathish |

| Layer                         | Technology                                                         | Owner   |
|-------------------------------|--------------------------------------------------------------------|---------|
|                               |                                                                    |         |
| <b>Scanners</b>               | Nuclei · Trivy · Gitleaks (subprocess)                             | Sathish |
| <b>Threat Intel</b>           | NVD · CISA KEV · FIRST EPSS APIs                                   | Sathish |
| <b>AI</b>                     | Anthropic Claude API (claude-sonnet-4-5)                           | Sathish |
| <b>Frontend Framework</b>     | Next.js 15 (App Router) + React 18 + TypeScript                    | Karthik |
| <b>UI Components</b>          | shadcn/ui (Radix primitives + Tailwind)                            | Karthik |
| <b>Styling</b>                | Tailwind CSS (utility-first)                                       | Karthik |
| <b>Frontend Auth</b>          | Auth.js v5 (NextAuth) — Credentials provider → FastAPI /auth/token | Karthik |
| <b>Typography</b>             | Inter font via next/font/google (self-hosted, zero CLS)            | Karthik |
| <b>Icons</b>                  | lucide-react (Feather fork, tree-shakeable)                        | Karthik |
| <b>File Upload</b>            | react-dropzone (drag-and-drop)                                     | Karthik |
| <b>Data Fetching</b>          | TanStack Query (caching, polling, retries)                         | Karthik |
| <b>Data Tables</b>            | TanStack Table (via shadcn DataTable)                              | Karthik |
| <b>Forms / Validation</b>     | react-hook-form + zod                                              | Karthik |
| <b>Charts</b>                 | Recharts (React-native, SSR-friendly)                              | Karthik |
| <b>Toasts / Notifications</b> | sonner                                                             | Karthik |
| <b>Dark Mode</b>              | next-themes                                                        | Karthik |
| <b>Containers</b>             | Docker Compose (dev) → Compose-prod with Caddy                     | Karthik |

| Layer      | Technology                                                        | Owner   |
|------------|-------------------------------------------------------------------|---------|
| CI / CD    | GitHub Actions (lint, typecheck, vitest, build images)            | Karthik |
| Monitoring | Sentry (@sentry/nextjs + Python SDK) + UptimeRobot + pg_dump cron | Karthik |
| Hosting    | Hetzner CX31 VPS + Caddy auto-SSL + Namecheap domain + Cloudflare | Karthik |

## MONTH 1 — Foundation & First Scans · Weeks 1-4

End goal: 3 scanners producing findings in a multi-tenant DB. Empty-but-functional Next.js UI.

### WEEK 1 — Shared setup & Data Store

Both pair daily this week. Shared foundation matters more than speed.

| Day   | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Done When                                                                                                                                                                |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Day 1 | <ul style="list-style-type: none"> <li>• Install Python 3.12, Docker Desktop, VS Code, git</li> <li>• Create GitHub repo with folders: backend/ frontend/ infra/</li> <li>• Add README.md skeleton + .gitignore (Python, Node, .env, .next/, dist/)</li> <li>• Run: docker run hello-world (confirm Docker works)</li> </ul>                                                                                                                                                                                     | <ul style="list-style-type: none"> <li>• Install Node.js 20 LTS (Next.js 15 requires ≥18.18; 20 is the LTS sweet spot)</li> <li>• Confirm: node -v · npm -v</li> <li>• Clone the repo Sathish just created</li> <li>• Inside frontend/: npx create-next-app@latest platform-ui --typescript --tailwind --app --src-dir --import-alias '@/*' --eslint</li> <li>• Verify: npm run dev → http://localhost:3000 shows Next.js starter</li> </ul>                                                                                                                                                                                                                                                                                                                   | <ul style="list-style-type: none"> <li>• Both run hello-world</li> <li>• Repo cloned</li> <li>• Next.js dev server loads at :3000</li> </ul>                             |
| Day 2 | <ul style="list-style-type: none"> <li>• Write docker-compose.yml — 3 services: postgres:16-alpine (5432), redis:7-alpine (6379), minio/minio (9000, 9001)</li> <li>• Run: docker compose up -d → confirm all 3 RUNNING</li> <li>• Write .env.example: DATABASE_URL, REDIS_URL, ANTHROPIC_API_KEY, JWT_SECRET, MINIO_USER, MINIO_PASSWORD</li> <li>• Add .env to .gitignore (NEVER commit .env)</li> </ul>                                                                                                       | <ul style="list-style-type: none"> <li>• Self-host Inter via next/font/google in app/layout.tsx (zero CLS, no external request)</li> <li>• Install icons: npm i lucide-react (Feather-style, tree-shakeable, single import path)</li> <li>• Install shadcn/ui base: npx shadcn@latest init (pick Slate, CSS variables, RSC: yes)</li> <li>• Add first components: npx shadcn@latest add button card input label dialog toast sonner</li> <li>• Configure tailwind.config.ts theme tokens (primary indigo, destructive red, warning amber)</li> <li>• Set .env.local with NEXT_PUBLIC_API_BASE_URL=http://localhost:8000</li> <li>• Scaffold App Router structure: app/(auth)/login, app/(app)/{dashboard,scans,scans/[id],findings,assets,settings}</li> </ul> | <ul style="list-style-type: none"> <li>• docker compose ps shows 3 green</li> <li>• .env.example committed</li> <li>• next build succeeds with Inter rendered</li> </ul> |
| Day 3 | <ul style="list-style-type: none"> <li>• cd backend/ · python -m venv .venv · pip install sqlalchemy alembic psycopg2-binary pydantic[email] python-dotenv</li> <li>• Write 5 SQLAlchemy models in app/models/: Org, User, Asset, Scan, Finding (with org_id FK on every table)</li> <li>• alembic init alembic · edit env.py to read DATABASE_URL · alembic revision --autogenerate -m 'initial'</li> <li>• alembic upgrade head · verify: docker exec -it &lt;pg&gt; psql -U user platform -c '\dt'</li> </ul> | <ul style="list-style-type: none"> <li>• Build app/(app)/layout.tsx — persistent shell: Sidebar (left, w-64) + TopBar (sticky, h-14)</li> <li>• Sidebar uses lucide-react: &lt;LayoutDashboard/&gt;, &lt;Search/&gt;, &lt;AlertTriangle/&gt;, &lt;Box/&gt;, &lt;Settings/&gt;</li> <li>• Use Next.js usePathname() for active-link styling (cn() utility from shadcn)</li> <li>• app/(auth)/layout.tsx is full-screen, NO shell — login is its own minimal page</li> <li>• Confirm all empty routes navigate cleanly (/login, /dashboard, /scans,</li> </ul>                                                                                                                                                                                                   | <ul style="list-style-type: none"> <li>• All 5 tables in Postgres</li> <li>• App shell renders with empty routes</li> <li>• Sidebar active states work</li> </ul>        |

| Day   | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                             | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Done When                                                                                                                                                                   |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |                                                                                                                                                                                                                                                                                                                                                                               | /findings, /assets, /settings)                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                             |
| Day 4 | <ul style="list-style-type: none"> <li>• Write app/database.py: engine = create_engine(DATABASE_URL, pool_pre_ping=True); SessionLocal; get_db() dep</li> <li>• Write seed script: insert 1 test org + 1 admin user (passlib bcrypt hashed password)</li> <li>• Verify: Python REPL → session.query(User).all() returns the user</li> <li>• Commit migration files</li> </ul> | <ul style="list-style-type: none"> <li>• Build LoginForm as a 'use client' component using shadcn Form + react-hook-form + zod</li> <li>• Email + password inputs, validation messages, submit button with loading state</li> <li>• No backend call yet — log to console on submit</li> <li>• Build Dashboard placeholder: grid of 4 stat cards (shadcn Card) showing '_'</li> <li>• Add Logout button in TopBar (lucide LogOut icon, just clears localStorage placeholder)</li> </ul> | <ul style="list-style-type: none"> <li>• Seed script inserts org+user</li> <li>• Login form validates, shows zod error messages</li> <li>• Logout button visible</li> </ul> |
| Day 5 | <ul style="list-style-type: none"> <li>• Integration: walk through schema with Karthik. Adjust column types if needed</li> <li>• Document schema in backend/SCHEMA.md with mermaid ER diagram</li> <li>• Set up pre-commit hooks: ruff + black for Python</li> <li>• Tag git: v0.0.1-data-store-ready</li> </ul>                                                              | <ul style="list-style-type: none"> <li>• Review schema with Sathish — confirm Finding fields cover everything UI needs</li> <li>• Set up pre-commit: ESLint (next/core-web-vitals) + Prettier + prettier-plugin-tailwindcss</li> <li>• Document component &amp; route structure in frontend/STRUCTURE.md</li> <li>• Add scripts: 'lint', 'format', 'typecheck' (tsc --noEmit) to package.json</li> <li>• Tag git: v0.0.1-frontend-skeleton-ready</li> </ul>                            | <ul style="list-style-type: none"> <li>• Schema reviewed</li> <li>• Pre-commit hooks block bad commits</li> <li>• Tag pushed</li> </ul>                                     |

## WEEK 2 — Backend skeleton (Sathish lead, Karthik wires Auth.js)

| Day   | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                    | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Done When                                                                                                                                                                                      |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Day 6 | <ul style="list-style-type: none"> <li>• pip install fastapi uvicorn[standard]</li> <li>• Write app/main.py: app = FastAPI(title='Security Platform v0.1'); CORS allow http://localhost:3000</li> <li>• GET /health → {status, version, db_ok: bool} (pings DB)</li> <li>• Add backend service to docker-compose.yml (build: ./backend, volumes for hot reload, depends_on)</li> </ul>                                                               | <ul style="list-style-type: none"> <li>• Install Auth.js v5: npm i next-auth@beta</li> <li>• Create app/api/auth/[...nextauth]/route.ts with CredentialsProvider</li> <li>• Provider authorize() POSTs to FastAPI /auth/token, returns JWT + user info</li> <li>• Configure JWT session strategy (no DB adapter needed — FastAPI is source of truth)</li> <li>• Build typed fetch wrapper lib/api.ts that attaches Authorization: Bearer &lt;token&gt; from session</li> </ul>                                                                         | <ul style="list-style-type: none"> <li>• GET /health from browser shows JSON, db_ok=true</li> <li>• Auth.js route handler responds at /api/auth/providers</li> </ul>                           |
| Day 7 | <ul style="list-style-type: none"> <li>• pip install python-jose[cryptography] passlib[bcrypt]</li> <li>• Write app/auth/jwt.py: create_access_token(data, exp=8h) + verify_token() + decode_token()</li> <li>• POST /auth/token (OAuth2PasswordRequestForm) → {access_token, token_type, expires_in}</li> <li>• GET /auth/me → returns current user from JWT (Depends(get_current_user))</li> </ul>                                                 | <ul style="list-style-type: none"> <li>• Wire LoginForm to call signIn('credentials', { email, password, redirect: false })</li> <li>• On success: router.push('/dashboard'); on error: sonner toast with backend error message</li> <li>• Create middleware.ts at root — protects (app)/* routes, redirects to /login if no session</li> <li>• Add 'Remember me' checkbox (sets maxAge 30d vs 8h in Auth.js session callback)</li> <li>• Confirm: visiting /dashboard while logged out → /login; while logged in → dashboard</li> </ul>               | <ul style="list-style-type: none"> <li>• POST /auth/token with seed user returns JWT</li> <li>• GET /auth/me returns user</li> <li>• UI login works end-to-end, middleware enforces</li> </ul> |
| Day 8 | <ul style="list-style-type: none"> <li>• Write Pydantic request/response schemas in app/schemas/: LoginRequest, TokenResponse, UserOut, OrgOut, AssetIn/Out, ScanIn/Out, FindingOut</li> <li>• Add POST /orgs (admin only), POST /users (admin only)</li> <li>• Write RBAC decorator: @require_role('admin' 'analyst' 'viewer')</li> <li>• CRITICAL: add org_id filter to EVERY query in every endpoint (helper: get_user_org_id dep)</li> </ul>     | <ul style="list-style-type: none"> <li>• Build /dashboard with 4 shadcn Cards: Total Scans · Open Critical · Open High · Assets</li> <li>• All cards show '—' for now (will wire later)</li> <li>• TopBar shows: user email · org name · lucide ChevronDown menu → Profile, Settings, Logout</li> <li>• Use shadcn DropdownMenu for the user menu</li> <li>• Sidebar nav: Dashboard · Scans · Findings · Assets · Settings (with active-state styling)</li> </ul>                                                                                      | <ul style="list-style-type: none"> <li>• Viewer role gets 403 on POST /scans</li> <li>• Admin can POST /orgs</li> <li>• Dashboard renders 4 cards</li> </ul>                                   |
| Day 9 | <ul style="list-style-type: none"> <li>• pip install celery[redis]</li> <li>• Write app/celery_app.py: broker_url=REDIS_URL, result_backend=REDIS_URL, task_routes</li> <li>• Write first task: @celery.task def run_scan_task(scan_id, target, scan_types, org_id) → time.sleep(5), update scan.status='complete'</li> <li>• Add worker service to docker-compose.yml: command: celery -A app.celery_app worker --loglevel=info -Q scans</li> </ul> | <ul style="list-style-type: none"> <li>• Install TanStack Table: npm i @tanstack/react-table — gives flexible headless tables (shadcn data-table wraps it)</li> <li>• Build ScansListPage (app/(app)/scans/page.tsx) — shadcn DataTable with columns Date · Target · Status · Findings · Actions</li> <li>• Use placeholder mock data for now (move to real fetch tomorrow)</li> <li>• Add 'New Scan' button (top right) that opens shadcn Dialog: NewScanDialog</li> <li>• NewScanDialog form (react-hook-form + zod): target URL input, 3</li> </ul> | <ul style="list-style-type: none"> <li>• Worker logs show task received + complete</li> <li>• Scans table renders mock data</li> <li>• Dialog opens</li> </ul>                                 |

| Day           | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                        | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Done When                                                                                                                                                                      |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               |                                                                                                                                                                                                                                                                                                                                                                                                                                                          | checkboxes (vuln, secret, sca)                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                |
| <b>Day 10</b> | <ul style="list-style-type: none"> <li>• Write scan endpoints: POST /scans (creates row, enqueues Celery task, returns {scan_id, eta}) · GET /scans (list, paginated) · GET /scans/{id} · GET /scans/{id}/findings</li> <li>• Store progress in Redis hash: scan:{id} → {status, progress_pct, current_step}</li> <li>• Worker updates this hash every step</li> <li>• Pydantic input validation: reject malformed URLs, IP ranges, repo URLs</li> </ul> | <ul style="list-style-type: none"> <li>• Install TanStack Query: npm i @tanstack/react-query — replaces RxJS-style polling</li> <li>• Wrap app/(app)/layout with QueryClientProvider</li> <li>• Wire ScansListPage useQuery(['scans']) → GET /scans (replace mock data)</li> <li>• Wire NewScanDialog submit → useMutation → POST /scans → invalidate query → close dialog</li> <li>• Wire row click → router.push(`/scans/\${id}`); build app/(app)/scans/[id]/page.tsx stub</li> </ul> | <ul style="list-style-type: none"> <li>• Real scan creates DB row + Celery task fires</li> <li>• Status moves queued→running→complete in 5s</li> <li>• UI refreshes</li> </ul> |

## WEEK 3 — First two scanners (Nuclei + Trivy) &amp; live scan UI

| Day    | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Done When                                                                                                                                                            |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Day 11 | <ul style="list-style-type: none"> <li>Download Nuclei binary:<br/>https://github.com/projectdiscovery/nuclei/releases (linux-amd64) → backend/tools/nuclei</li> <li>chmod +x · run manually: ./nuclei -target https://example.com -t cves -json -silent</li> <li>Pipe output to file · study JSON structure (one finding per line, JSONL format)</li> <li>Document field mapping: template-id → finding.title, info.severity → finding.severity, matched-at → finding.url</li> </ul> | <ul style="list-style-type: none"> <li>Build &lt;SeverityBadge severity={...}/&gt; component using shadcn Badge with variant prop</li> <li>Color-coded: critical=red, high=orange, medium=amber, low=blue, info=slate</li> <li>Use across the app — wherever severity is shown</li> <li>Add Storybook-style demo route /dev/badges (only in NODE_ENV=development) for visual QA</li> <li>Write quick test with Vitest: render &lt;SeverityBadge severity='critical'/&gt; → has class bg-red-</li> </ul>  | <ul style="list-style-type: none"> <li>Nuclei JSON understood</li> <li>Badge component renders all 5 severities</li> <li>Vitest passes</li> </ul>                    |
| Day 12 | <ul style="list-style-type: none"> <li>Write app/adapters/base.py: BaseAdapter ABC with run(target, options) → list[Finding] abstract method</li> <li>Write app/adapters/nuclei.py: NucleiAdapter(BaseAdapter), subprocess.run with timeout=600s</li> <li>Parse stdout line by line as JSON → Finding Pydantic model</li> <li>Handle: timeout, non-zero exit, partial JSON (skip malformed lines)</li> </ul>                                                                          | <ul style="list-style-type: none"> <li>Build FindingsTable using shadcn DataTable (TanStack Table) — cols CVE · Severity · Title · Asset · Tool · Discovered</li> <li>Default sort: severity desc (use TanStack's getSortedRowModel)</li> <li>Wire to GET /scans/{id}/findings with TanStack Query; refetchInterval: 5000 while scan.status !== 'complete'</li> <li>When complete, set refetchInterval to false (stops polling)</li> </ul>                                                               | <ul style="list-style-type: none"> <li>Nuclei adapter returns list[Finding]</li> <li>Findings appear in UI within 5s</li> <li>Polling stops on completion</li> </ul> |
| Day 13 | <ul style="list-style-type: none"> <li>Install Trivy: apt install trivy (or download binary) into backend/tools/trivy</li> <li>Test: trivy image --format json --severity HIGH,CRITICAL nginx:1.24 &gt; out.json</li> <li>Write app/adapters/trivy.py: TrivyAdapter — parse Results[].Vulnerabilities[] → list[Finding]</li> <li>Extract fixed_version and store in Finding.metadata JSONB column</li> </ul>                                                                          | <ul style="list-style-type: none"> <li>Add 'Docker image' input field to NewScanDialog (conditional render when SCA checkbox is checked — use react-hook-form watch())</li> <li>zod schema validates image format: regex ^[a-z0-9-/_.:@]+\$</li> <li>In FindingsTable, show 'Fix: x.y.z' shadcn Badge next to CVE when tool === 'trivy' &amp;&amp; metadata.fixed_version</li> <li>Add shadcn ToggleGroup above table: All · Critical · High · Medium · Low (controls TanStack column filter)</li> </ul> | <ul style="list-style-type: none"> <li>Trivy scan of nginx:1.24 yields 20+ CVEs</li> <li>Fix-version badge visible</li> <li>Filter chips work</li> </ul>             |
| Day 14 | <ul style="list-style-type: none"> <li>Wire Nuclei + Trivy into run_scan_task with Celery group (parallel): group(nuclei.s(...), trivy.s(...)).apply_async()</li> <li>Use chord to collect results and write to Postgres in single transaction</li> <li>Each Finding gets org_id from scan.org_id (CRITICAL: never trust user input)</li> <li>Add try/except around each adapter — one tool failing must NOT kill the whole scan</li> </ul>                                           | <ul style="list-style-type: none"> <li>On scan detail page, add SSE consumer: const es = new EventSource('/api/scans/{id}/progress') in useEffect inside a 'use client' component</li> <li>Backend endpoint GET /scans/{id}/progress (text/event-stream) pushes progress every 10s</li> <li>Show shadcn Progress bar (0–100%) + current step text</li> <li>Fallback: if EventSource fails (es.onerror), TanStack Query refetchInterval kicks in</li> </ul>                                               | <ul style="list-style-type: none"> <li>Multi-tool scan runs in parallel</li> <li>Progress bar updates live via SSE</li> <li>Tool crash doesn't kill scan</li> </ul>  |
| Day 15 | <ul style="list-style-type: none"> <li>Write Postman/Bruno collection covering all endpoints</li> <li>Write pytest integration tests: test_full_scan_lifecycle (queue → run → fetch findings → 200)</li> <li>Test org isolation: Org A user GET /scans/{org_b_scan} → 403</li> </ul>                                                                                                                                                                                                  | <ul style="list-style-type: none"> <li>Polish: empty state for ScansList ('No scans yet — start your first scan' with CTA button)</li> <li>Loading skeletons (shadcn Skeleton component) for table while data loads</li> </ul>                                                                                                                                                                                                                                                                           | <ul style="list-style-type: none"> <li>Tests green</li> <li>Org isolation enforced</li> <li>UX clean</li> </ul>                                                      |

| Day | Sathish — Backend / Security / AI                                          | Karthik — Frontend / Deploy                                                                                                                                       | Done When |
|-----|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
|     | <ul style="list-style-type: none"><li>• Tag: v0.1.0-two-scanners</li></ul> | <ul style="list-style-type: none"><li>• Error toasts: any 4xx/5xx from backend → sonner toast with user-friendly message</li><li>• Tag: v0.1.0-ui-week3</li></ul> |           |

## WEEK 4 — Third scanner (Gitleaks) + Finding detail panel + hardening

| Day    | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Done When                                                                                                                                       |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Day 16 | <ul style="list-style-type: none"> <li>Download Gitleaks binary → backend/tools/gitleaks</li> <li>Write app/adapters/gitleaks.py: clone repo to /tmp/scan_{scan_id} (shallow: --depth 50), run gitleaks detect --report-format json</li> <li>Parse findings: secret_type, file, line, commit, partial_secret (mask middle chars)</li> <li>Cleanup: shutil.rmtree(/tmp/scan_{id}) in finally block</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                            | <ul style="list-style-type: none"> <li>Add 'Git repo URL' input to NewScanDialog (conditional render when Secret Scan checked)</li> <li>zod validates: ^https://(github gitlab bitbucket)\.com/.+\.git\$</li> <li>Show file path + commit hash in FindingDetailPanel (next day)</li> <li>Add an icon legend (lucide icons) below scan-type checkboxes: which input field goes with which scan type</li> </ul>                                                                                                                                                                        | <ul style="list-style-type: none"> <li>Gitleaks scans a test repo</li> <li>Repo URL validation works</li> <li>Temp folder cleaned up</li> </ul> |
| Day 17 | <ul style="list-style-type: none"> <li>Add deduplication: same secret-fingerprint (hash of normalized secret) across N commits = 1 finding</li> <li>Add fp_candidate boolean field (will use Week 6 for FP filtering)</li> <li>Wire Gitleaks into Celery group with Nuclei + Trivy — full 3-way parallel scan works</li> <li>Add scan timeout safety: chord with countdown, mark stuck scans as failed after 30 min</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                          | <ul style="list-style-type: none"> <li>Build FindingDetailPanel using shadcn Sheet (right side, w-[480px])</li> <li>Sections: Summary · Severity · Asset · Tool source · Discovered date · Raw Output (shadcn Collapsible)</li> <li>Click row in FindingsTable → setOpen(true) + setSelectedFinding(row)</li> <li>Close on X button or onOpenChange(false)</li> </ul>                                                                                                                                                                                                                | <ul style="list-style-type: none"> <li>3-tool parallel scan works</li> <li>Sheet opens on row click</li> <li>Raw output collapsible</li> </ul>  |
| Day 18 | <ul style="list-style-type: none"> <li>Add rate limiting: pip install slowapi · @limiter.limit('10/hour') on POST /scans</li> <li>Add input sanitization: reject targets matching ^(127\. 10\. 192\.168 169\.254) (no localhost/RFC1918 scans)</li> <li>Add user-agent on outbound tool calls so target server logs identify us</li> <li>FR-SCN-13 — Exploit validation mode: for findings tagged severity=critical, run Nuclei with -validate flag to confirm exploitability (not just signature match)</li> <li>Add finding.exploit_validated boolean column; migration</li> <li>Validated findings get +0.5 boost to priority_score; unvalidated 'critical' findings get -0.2 (likely false confidence)</li> <li>Add audit log table: every POST /scans writes {user_id, org_id, target, ts} to audit_log</li> </ul> | <ul style="list-style-type: none"> <li>Build /settings page with shadcn Tabs: API Tokens · Org Info (read-only) · Notifications</li> <li>Add green 'CONFIRMED EXPLOITABLE' shadcn Badge next to validated findings (sits beside severity badge)</li> <li>Tooltip on badge: 'Vulnerability confirmed reachable + exploitable via active probe'</li> <li>User profile widget in TopBar: shadcn Avatar with initials, DropdownMenu with Profile · Settings · Logout</li> <li>Style polish: consistent spacing (Tailwind's space-y utilities), hover states (hover:bg-accent)</li> </ul> | <ul style="list-style-type: none"> <li>Rate limit returns 429</li> <li>Validated findings show badge</li> <li>Audit log populated</li> </ul>    |
| Day 19 | <ul style="list-style-type: none"> <li>Asset management: POST /assets, GET /assets, PATCH /assets/{id} (set asset_weight: 0.5/1.0/1.5/2.0)</li> <li>Auto-register asset on first scan if target not in assets table</li> <li>Link Finding.asset_id → Asset.id (FK)</li> <li>Migration: alembic revision -m 'add asset_weight + audit_log'</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <ul style="list-style-type: none"> <li>Build /assets page: shadcn DataTable of org's assets · Name · Type · Weight · Last scan · Findings count</li> <li>Inline edit weight using shadcn Select (Low/Medium/High/Critical) → useMutation PATCH</li> <li>Build AddAssetDialog (manual creation, also auto-creates on scan)</li> <li>Wire dashboard 'Assets' card to GET /assets?count_only=true</li> </ul>                                                                                                                                                                            | <ul style="list-style-type: none"> <li>Assets endpoint works</li> <li>Inline weight edit saves</li> <li>Dashboard count correct</li> </ul>      |
| Day 20 | <ul style="list-style-type: none"> <li>Multi-tenant security audit: create 2 test orgs, run 5 scans on each, attempt cross-tenant access on EVERY endpoint</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <ul style="list-style-type: none"> <li>Cross-browser test: Chrome, Firefox, Safari (if available), Edge</li> <li>Mobile responsive pass: dashboard + scans list + finding detail on iPhone</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                | <ul style="list-style-type: none"> <li>Zero cross-tenant leaks</li> </ul>                                                                       |

| Day | Sathish — Backend / Security / AI                                                                                                                                                                                                               | Karthik — Frontend / Deploy                                                                                                                                                                                                   | Done When                                                                           |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|     | <ul style="list-style-type: none"><li>• Fix any gap found (most likely places: GET /findings/{id}, GET /assets/{id})</li><li>• Write pytest fixture for isolation tests so they run on every CI</li><li>• Tag: v0.1.0-month1-complete</li></ul> | <ul style="list-style-type: none"><li>width (375px)</li><li>• Use Tailwind responsive prefixes (sm:, md:, lg:) — collapse sidebar to drawer (shadcn Sheet) on &lt;md</li><li>• Tag: v0.1.0-month1-complete-frontend</li></ul> | <ul style="list-style-type: none"><li>• Mobile works</li><li>• Tag pushed</li></ul> |

## MONTH 2 — Intelligence & AI Layer · Weeks 5–8

End goal: AI-enriched findings with source-call reachability, agentic chat, fix PRs, single-screen CISO view, and Jira/Slack/PDF outputs.

### WEEK 5 — Threat intelligence enrichment (NVD · KEV · EPSS)

| Day           | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                             | Done When                                                                                                                                            |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Day 21</b> | <ul style="list-style-type: none"> <li>Write app/intel/nvd.py: NVDClient with GET <a href="https://services.nvd.nist.gov/rest/json/cves/2.0?cveId={id}">https://services.nvd.nist.gov/rest/json/cves/2.0?cveId={id}</a></li> <li>Extract: cvss_v3.baseScore, cvss_v3.vectorString, description, cwe[] from configurations</li> <li>Cache in Redis: SET nvd:{cve_id} &lt;json&gt; EX 86400 (24h TTL)</li> <li>Get NVD API key (free): <a href="https://nvd.nist.gov/developers/request-an-api-key">nvd.nist.gov/developers/request-an-api-key</a> — raises rate limit</li> </ul> | <ul style="list-style-type: none"> <li>Add cvss_score column to FindingsTable (sortable via TanStack)</li> <li>Add CVSS vector string tooltip on hover (shadcn Tooltip)</li> <li>Add CWE tag chips in FindingDetailPanel (shadcn Badge, e.g. CWE-89 SQL Injection)</li> <li>Add column show/hide toggle using shadcn DropdownMenuCheckboxItem (TanStack's columnVisibility API)</li> </ul>                              | <ul style="list-style-type: none"> <li>NVD data in Postgres</li> <li>Cache hit confirmed (&lt;1ms 2nd call)</li> <li>CVSS column sortable</li> </ul> |
| <b>Day 22</b> | <ul style="list-style-type: none"> <li>Write app/intel/kev.py: download CISA KEV JSON (<a href="https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json">https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json</a>)</li> <li>Parse vulnerabilities[].cveID → load into Redis SET: SADD kev:cves CVE-2024-X</li> <li>Lookup: SISMEMBER kev:cves CVE-XXX (O(1))</li> <li>Celery Beat task: refresh daily at 02:00 UTC</li> </ul>                                                                                        | <ul style="list-style-type: none"> <li>Add 'EXPLOITED IN WILD' red Badge to KEV findings (next to severity)</li> <li>Show KEV due_date in FindingDetailPanel (CISA's remediation deadline)</li> <li>Always sort KEV findings to top regardless of severity sort (custom sortingFn in TanStack)</li> <li>Add KEV filter ToggleGroup item above table</li> </ul>                                                          | <ul style="list-style-type: none"> <li>KEV badge appears</li> <li>Refresh runs daily</li> <li>KEV findings top</li> </ul>                            |
| <b>Day 23</b> | <ul style="list-style-type: none"> <li>Write app/intel/epss.py: EPSSClient batch up to 100 CVEs per call to <a href="https://api.first.org/data/v1/epss?cve=A,B,C">https://api.first.org/data/v1/epss?cve=A,B,C</a></li> <li>Cache: SET epss:{cve_id} &lt;score&gt; EX 86400</li> <li>Add epss_score (float 0–1) and epss_percentile to Finding</li> <li>Migration for new columns</li> </ul>                                                                                                                                                                                   | <ul style="list-style-type: none"> <li>Add EPSS percentile bar in FindingDetailPanel: shadcn Progress with custom color via className (green 0–30%, amber 30–70%, red 70–100%)</li> <li>Tooltip explains: 'Probability this CVE is exploited in the wild in next 30 days'</li> <li>Add EPSS column to table (sortable, formatted as percentile)</li> <li>Quick filter: 'High EPSS' chip (&gt;70%)</li> </ul>            | <ul style="list-style-type: none"> <li>EPSS in DB</li> <li>Batch fetch confirmed</li> <li>Color bar renders</li> </ul>                               |
| <b>Day 24</b> | <ul style="list-style-type: none"> <li>Write enrich_findings_task: Celery chord called after scan completes — enrich all findings in batches</li> <li>Order: NVD lookup → KEV check → EPSS lookup → write back to DB</li> <li>Compute priority_score = round(epss × cvss × (2 if kev else 1) × asset_weight, 3)</li> <li>Store priority_score + priority_rank (window function in SQL)</li> </ul>                                                                                                                                                                               | <ul style="list-style-type: none"> <li>Make priority_score the default sort in FindingsTable</li> <li>Add rank badge (#1, #2, #3...) for top 10 findings — small shadcn Badge variant</li> <li>Build 'Top 10 — Fix Immediately' section at top of ScanDetail (above the full findings table)</li> <li>Use shadcn Card grid (1 col mobile, 2 col tablet, prominent severity + asset + AI summary placeholder)</li> </ul> | <ul style="list-style-type: none"> <li>Findings auto-enriched</li> <li>Top 10 ranked correctly</li> </ul>                                            |
| <b>Day 25</b> | <ul style="list-style-type: none"> <li>Integration test: scan a known-vulnerable target (e.g. test app with old</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                      | <ul style="list-style-type: none"> <li>Build IntelStatusCard on Dashboard (shadcn Card): last NVD/KEV/EPSS</li> </ul>                                                                                                                                                                                                                                                                                                   | <ul style="list-style-type: none"> <li>100 findings enriched</li> </ul>                                                                              |

| Day | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                             | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                   | Done When                                                                                    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
|     | <p>jQuery), confirm CVSS+KEV+EPSS+rank all populated</p> <ul style="list-style-type: none"><li>• Performance test: 100-finding scan completes enrichment in &lt;60s</li><li>• Add basic retry on intel API failures (3x with exponential backoff)</li><li>• Tag: v0.2.0-intel-ready</li></ul> | <p>refresh times + Redis cache sizes</p> <ul style="list-style-type: none"><li>• Useful for debugging + reassures users data is fresh</li><li>• Add 'Force refresh' button (admin only, role check via session) → POST /admin/intel/refresh</li><li>• Tag frontend: v0.2.0-intel-ui</li></ul> | <p>in &lt;60s</p> <ul style="list-style-type: none"><li>• Intel status panel works</li></ul> |

**WEEK 6 — Claude AI integration: enrichment, reachability (FR-AI-08), agentic chat (FR-AI-10)**

| Day           | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Done When                                                                                                                                                                                      |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Day 26</b> | <ul style="list-style-type: none"> <li>• Sign up at console.anthropic.com → create API key → add to .env: ANTHROPIC_API_KEY=sk-ant-...</li> <li>• pip install anthropic</li> <li>• Write app/ai/claude_client.py: wrapper around client.messages.create(model='claude-sonnet-4-5', max_tokens=1000, temperature=0)</li> <li>• Test: send one CVE context, print response</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                        | <ul style="list-style-type: none"> <li>• Build AI Summary section in FindingDetailPanel (initial placeholder text)</li> <li>• Two subsections: 'Plain English Explanation' and 'Remediation Steps' (numbered shadcn ol)</li> <li>• Add loading state with shadcn Skeleton while AI call runs</li> <li>• Add 'Regenerate' button (admin only) — calls backend to re-run AI</li> </ul>                                                                                                                                             | <ul style="list-style-type: none"> <li>• Claude API returns response</li> <li>• Detail panel shows AI placeholders</li> </ul>                                                                  |
| <b>Day 27</b> | <ul style="list-style-type: none"> <li>• Write app/ai/prompt_builder.py: build_prompt(finding) injects CVE + CVSS + EPSS + KEV + asset + tool</li> <li>• System prompt: 'Return ONLY valid JSON, no markdown, no preamble.' with exact schema</li> <li>• Schema fields: plain_english, remediation_steps[], severity_override, override_reason, jira_ticket{summary, description, priority, labels}, exec_summary_line</li> <li>• Parse with Pydantic v2 AIEnrichment model; on parse failure, retry once with stricter prompt</li> </ul>                                                                                                                                                                                                                                                                                  | <ul style="list-style-type: none"> <li>• Wire FindingDetailPanel to useQuery(['ai-enrichment', findingId]) → GET /findings/{id}/ai-enrichment</li> <li>• Render plain_english as paragraph (prose tailwind classes), remediation_steps as &lt;ol&gt;</li> <li>• If severity_override present: shadcn Alert variant 'destructive' — 'Severity escalated: HIGH → CRITICAL (reason)'</li> <li>• Cache AI result on Finding row server-side — never re-call for same finding (TanStack staleTime: Infinity)</li> </ul>               | <ul style="list-style-type: none"> <li>• AI summary renders</li> <li>• Severity override visible</li> <li>• No re-call on 2nd view</li> </ul>                                                  |
| <b>Day 28</b> | <ul style="list-style-type: none"> <li>• FR-AI-08 — Source-call reachability for SCA findings (DefectDojo CANNOT do this — they don't hold source)</li> <li>• For each Trivy CVE: parse repo (already cloned for Gitleaks) with tree-sitter or simple AST; extract imports + call sites of the vulnerable package</li> <li>• Build reachability_prompt: pass CVE description + vulnerable function names + repo import graph + N call-site snippets</li> <li>• Claude classifies: reachable   unreachable   uncertain + confidence + reasoning</li> <li>• Add finding.reachability column (enum), reachability_reason text; migration</li> <li>• Drop priority_score by 0.7× for 'unreachable'; bump 1.2× for 'reachable'; keep as-is for 'uncertain'</li> <li>• Cost tracking still active: log ai_calls table</li> </ul> | <ul style="list-style-type: none"> <li>• Add ReachabilityBadge component: green 'REACHABLE' / gray 'NOT REACHED IN CODE' / amber 'UNCERTAIN'</li> <li>• Show badge inline in FindingsTable for SCA findings (column after severity)</li> <li>• In FindingDetailPanel, add 'Reachability Analysis' section under AI Summary</li> <li>• Show the call-site snippet + Claude's reasoning in a shadcn Collapsible</li> <li>• Add table filter: 'Reachable only' shadcn Toggle — defaults OFF (most users want to see all)</li> </ul> | <ul style="list-style-type: none"> <li>• 50 findings → reachability analysis in &lt;60s</li> <li>• Unreachable CVEs sort below reachable ones</li> <li>• Reasoning visible in panel</li> </ul> |
| <b>Day 29</b> | <ul style="list-style-type: none"> <li>• FR-AI-10 — Ask-a-question agentic chat over scan data (defines AI-first positioning)</li> <li>• Build app/ai/agent.py: Claude with tool_use giving access to: list_findings(filters), get_finding(id), get_scan(id), get_asset(id), search_cves(query)</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <ul style="list-style-type: none"> <li>• Build AskAgent component as floating action button (Lucide MessageCircleQuestion) bottom-right of ScanDetail</li> <li>• On click: opens shadcn Sheet (right side) with chat UI — message list + input + suggested questions</li> <li>• Suggested questions surface from prompt examples: 'What should I fix</li> </ul>                                                                                                                                                                  | <ul style="list-style-type: none"> <li>• Agent answers 5 sample questions correctly</li> <li>• Streaming UI works</li> <li>• Citations link to</li> </ul>                                      |

| Day    | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Done When                                                                                                                                   |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
|        | <ul style="list-style-type: none"> <li>Each tool returns minimal JSON; agent loops up to 5 tool calls per question</li> <li>Endpoint POST /scans/{id}/ask {question} → returns {answer, citations: [finding_id, ...]}</li> <li>Examples: 'What changed since last scan?', 'What should I fix this sprint?', 'Why is CVE-2024-X critical?'</li> <li>Wire ai_enrichment_task: after intel enrichment, batch findings (5 per batch) and call Claude in parallel for plain-English summaries</li> <li>Cost tracking: log {scan_id, input_tokens, output_tokens, call_type} per call to ai_calls table (call_type: 'enrich' or 'agent')</li> </ul> | <p>this sprint?', 'What changed since last scan?'</p> <ul style="list-style-type: none"> <li>Streaming responses (fetch with ReadableStream, parse SSE)</li> <li>Citations render as clickable shadow Badge → opens FindingDetailPanel</li> </ul>                                                                                                                                                                                                                                                                                                             | findings                                                                                                                                    |
| Day 30 | <ul style="list-style-type: none"> <li>End-to-end test: real scan target → expect CVSS, EPSS, KEV, AI summary, reachability, agent answers, all visible</li> <li>Performance: 50-finding scan completes full enrichment (intel + AI + reachability) in &lt;3 min</li> <li>Resilience: kill Redis mid-scan, confirm scan retries gracefully</li> <li>Add /admin/ai-usage endpoint: cost per org per month (input/output tokens, by call_type)</li> <li>Tag: v0.3.0-ai-enrichment-ready</li> </ul>                                                                                                                                              | <ul style="list-style-type: none"> <li>Build /admin/ai-usage page: monthly token usage + estimated cost</li> <li>Install Recharts (npm i recharts): bar chart input/output tokens per day; line chart cumulative cost</li> <li>Stack bars by call_type (enrich vs agent) so you can see which feature drives spend</li> <li>Polish FindingDetailPanel: shadow Tabs for Summary / Remediation / Reachability / Raw</li> <li>Add keyboard shortcuts (react-hotkeys-hook): J/K navigate findings, Esc close panel</li> <li>Tag frontend: v0.3.0-ai-ui</li> </ul> | <ul style="list-style-type: none"> <li>Full enriched scan demo-able</li> <li>Cost dashboard accurate</li> <li>Keyboard nav works</li> </ul> |

## WEEK 7 — Outputs: Jira, Slack, AI-generated fix PRs (FR-OUT-04), PDF

| Day    | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Done When                                                                                                                                                                                        |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Day 31 | <ul style="list-style-type: none"> <li>• pip install atlassian-python-api (cleaner than legacy 'jira' lib)</li> <li>• Add JiraConfig table: org_id, base_url, project_key, email, api_token (encrypted at rest with Fernet)</li> <li>• POST /findings/{id}/create-jira-ticket: uses AI-generated jira_ticket fields, returns {jira_url, issue_key}</li> <li>• Store jira_issue_key on Finding row; disable button if already created</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | <ul style="list-style-type: none"> <li>• Build /settings → Jira tab: form (react-hook-form + zod) for base_url (https://yourco.atlassian.net), project key (e.g. SEC), email, API token</li> <li>• 'Test connection' button → POST /admin/jira/test → 200 or descriptive sonner toast</li> <li>• In FindingDetailPanel: 'Create Jira Ticket' button (disabled if jira config not set OR jira_issue_key already present)</li> <li>• After creation: show 'Jira: SEC-1234' as a Link (lucide ExternalLink icon), disable button</li> </ul>          | <ul style="list-style-type: none"> <li>• Jira ticket created with AI content</li> <li>• Link opens ticket</li> </ul>                                                                             |
| Day 32 | <ul style="list-style-type: none"> <li>• pip install slack-sdk</li> <li>• Add SlackConfig table: org_id, webhook_url (encrypted), critical_only (bool), channel</li> <li>• Celery task post_slack_summary(scan_id): on scan completion, post if any CRITICAL or KEV findings</li> <li>• Slack message: 'X CRITICAL findings on &lt;target&gt; · Top 3: ...' with link to scan</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <ul style="list-style-type: none"> <li>• Build /settings → Slack tab: webhook URL input, channel name, shadcn Switch 'critical findings only'</li> <li>• 'Send test message' button → POST /admin/slack/test → posts test message</li> <li>• Add notification preferences to user profile (email digest, in-app only, both — placeholder UI for now)</li> <li>• Show 'Last Slack alert sent: 2 min ago' badge on settings</li> </ul>                                                                                                              | <ul style="list-style-type: none"> <li>• Slack alert on critical scan</li> <li>• Critical-only toggle respected</li> </ul>                                                                       |
| Day 33 | <ul style="list-style-type: none"> <li>• FR-OUT-04 — AI-generated fix PRs (the demo 'wow' moment; DefectDojo creates tickets, we create commits)</li> <li>• pip install PyGithub (also GitLab via python-gitlab)</li> <li>• Add GitHubConfig table: org_id, installation_id, access_token (encrypted Fernet); OAuth App or GitHub App credentials</li> <li>• POST /findings/{id}/create-fix-pr endpoint with autofix logic per finding type: <ul style="list-style-type: none"> <li>• - Gitleaks: clone, remove secret from file, add to .gitignore, write PR body with rotation steps</li> <li>• - Trivy SCA: parse package.json/requirements.txt/go.mod, bump to fixed_version, run lockfile update</li> <li>• - Trivy misconfig (Dockerfile/IaC): Claude generates the patch, validate diff size &lt;50 lines before applying</li> </ul> </li> <li>• Always open as branch fix/{finding-id} → PR with AI-written description</li> <li>• Store pr_url, pr_status on Finding row</li> </ul> | <ul style="list-style-type: none"> <li>• Add 'Create Fix PR' button next to 'Create Jira Ticket' in FindingDetailPanel</li> <li>• Show as primary action for Gitleaks + SCA findings (auto-fixable types)</li> <li>• Disabled for findings where autofix not supported (e.g. web app vulns); tooltip explains why</li> <li>• After PR created: show 'PR #142 opened' as a Link (lucide GitPullRequest) with status badge (open/merged/closed)</li> <li>• Build /settings → GitHub tab: install button → OAuth flow → store credentials</li> </ul> | <ul style="list-style-type: none"> <li>• Gitleaks finding → PR removes secret + .gitignore update</li> <li>• Trivy SCA finding → PR bumps version</li> <li>• PR status reflects merge</li> </ul> |
| Day 34 | <ul style="list-style-type: none"> <li>• Executive summary: single Claude call AFTER all findings enriched</li> <li>• Input: top 10 findings (with AI summaries + reachability) + scan metadata + counts by severity</li> <li>• Output: 3–5 sentence plain-English summary for CISO/exec audience</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <ul style="list-style-type: none"> <li>• Add Executive Summary shadcn Card at top of ScanDetail (above Top 10), text-base leading-relaxed</li> <li>• Add 'Copy' button (lucide Copy icon) on summary</li> <li>• Add 'Download Report' button (top right, primary variant)</li> </ul>                                                                                                                                                                                                                                                              | <ul style="list-style-type: none"> <li>• Exec summary generates after scan</li> <li>• PDF downloads, renders in Adobe + Preview + Chrome</li> </ul>                                              |

| Day    | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Done When                                                                                                                                                                               |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|        | <ul style="list-style-type: none"> <li>• Store on Scan.exec_summary; cached, never re-generated</li> <li>• pip install reportlab</li> <li>• Build app/reports/pdf.py: ScanReport class with sections — Cover · Executive Summary · Top 10 (AI summaries + remediation + reachability) · Full list</li> <li>• Endpoint POST /scans/{id}/export?template=executive technical → returns PDF bytes</li> <li>• Store PDF in MinIO: scans/{org_id}/{scan_id}.pdf for re-download</li> </ul>                                                                                                                       | <ul style="list-style-type: none"> <li>• On click: show loading toast (sonner.loading), trigger browser download via blob URL</li> <li>• 'Report template' select in dialog: executive (default) or technical</li> <li>• Print-friendly CSS in globals.css: @media print { hide nav, expand details }</li> </ul>                                                                                                                                                                                                              |                                                                                                                                                                                         |
| Day 35 | <ul style="list-style-type: none"> <li>• Per-finding Slack thread (extends Day 32 basic Slack alert)</li> <li>• For each critical/KEV finding: open a Slack thread tagging asset_owner (from Asset.owner_email field)</li> <li>• Thread body includes AI summary + reachability + 'Create Fix PR' button link + 'Create Jira' button link</li> <li>• Slash commands placeholder: /fixed and /false-positive (full handler in v1.1)</li> <li>• Output integration test: end-to-end flow — scan → Slack thread per critical → PR via button → Jira via button</li> <li>• Tag: v0.4.0-outputs-ready</li> </ul> | <ul style="list-style-type: none"> <li>• Add /settings → Assets tab: bulk-set Asset.owner_email (CSV upload via react-dropzone)</li> <li>• react-dropzone: drag CSV → parse client-side with papaparse → preview table → confirm → POST /assets/bulk</li> <li>• Polish all output buttons in FindingDetailPanel: consistent loading states (sonner toasts), success/error feedback</li> <li>• Mobile-responsive check on all /settings tabs (Tailwind sm: md: prefixes)</li> <li>• Tag frontend: v0.4.0-outputs-ui</li> </ul> | <ul style="list-style-type: none"> <li>• Critical finding → Slack thread w/ owner tagged</li> <li>• CSV asset owner upload works</li> <li>• All output actions feel polished</li> </ul> |

## WEEK 8 — Hardening, single-screen CISO view (FR-UI-08), multi-tenant audit

| Day           | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Done When                                                                                                                                              |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Day 36</b> | <ul style="list-style-type: none"> <li>Run 5 concurrent scans on dev. Watch: docker stats, slow query log, Celery queue depth</li> <li>Identify bottlenecks: usually NVD rate limit or Postgres slow queries on findings table</li> <li>Add indexes: (org_id, scan_id), (org_id, severity), (cve_id, priority_score DESC)</li> <li>Run alembic migration</li> </ul>                                                                                                                                                                                                                                                                                                                                                       | <ul style="list-style-type: none"> <li>Lighthouse audit on /scans and /scans/[id]: aim Performance &gt;85, Accessibility &gt;90</li> <li>Next.js wins here: next/image for images, automatic font optimization, App Router code splitting</li> <li>Fix low-hanging issues: add aria-label to icon-only buttons (shadcn already does sr-only labels for most)</li> <li>Test with axe DevTools; document accessibility status in frontend/ACCESSIBILITY.md</li> </ul>                                                                                                                                                                                            | <ul style="list-style-type: none"> <li>5 parallel scans complete in &lt;3 min</li> <li>Lighthouse Perf &gt;85, A11y &gt;90</li> </ul>                  |
| <b>Day 37</b> | <ul style="list-style-type: none"> <li>Multi-tenant audit (deep): script that brute-checks every endpoint with cross-org IDs</li> <li>Tests: GET /findings/{id} of Org B by Org A user → must 403, never 200</li> <li>Add hypothesis-based property test: 'for any org_a, org_b, no Org A query returns Org B rows'</li> <li>Document audit in SECURITY.md</li> </ul>                                                                                                                                                                                                                                                                                                                                                     | <ul style="list-style-type: none"> <li>Polish all forms: consistent labels, helper text via shadcn FormDescription, validation via FormMessage</li> <li>Add 'unsaved changes' guard on Settings pages: use beforeunload + Next.js useEffect cleanup</li> <li>Add global error boundary: app/global-error.tsx → friendly screen with 'Report bug' button</li> <li>Verify dark mode via next-themes (npm i next-themes) — wire ThemeProvider, ModeToggle in TopBar</li> </ul>                                                                                                                                                                                    | <ul style="list-style-type: none"> <li>Zero cross-tenant leaks (reconfirmed)</li> <li>Forms feel professional</li> <li>Dark mode usable</li> </ul>     |
| <b>Day 38</b> | <ul style="list-style-type: none"> <li>Add finding tag system: tag findings with custom labels (e.g. 'tech-debt', 'compliance', 'q3-priority')</li> <li>POST /findings/{id}/tags, DELETE /findings/{id}/tags/{name}; many-to-many via finding_tags</li> <li>Tag-based filters in GET /findings?tag=x</li> <li>Index on (org_id, tag) for fast lookup</li> </ul>                                                                                                                                                                                                                                                                                                                                                           | <ul style="list-style-type: none"> <li>Add tag chips in FindingsTable row + FindingDetailPanel (shadcn Badge)</li> <li>Add 'Add tag' inline action: shadcn Combobox with autocomplete from org's existing tags</li> <li>Filter dropdown above table: multi-select tags (TanStack filterFn)</li> <li>Tag color rotation: deterministic hash → one of 8 muted colors (consistent across renders)</li> </ul>                                                                                                                                                                                                                                                      | <ul style="list-style-type: none"> <li>Tag CRUD works</li> <li>Filter by tag returns correct subset</li> <li>Tag colors consistent</li> </ul>          |
| <b>Day 39</b> | <ul style="list-style-type: none"> <li>FR-UI-08 — Single-screen CISO view (the sales-call asset; one URL CISO shows the board)</li> <li>Build /executive endpoint: GET /executive returns {posture_score (0-100), top_risks: [...], wow_trend: {fixed, new, regressed}, exec_summary_paragraph, kev_count, validated_critical_count}</li> <li>posture_score = weighted formula: -10/critical, -3/high, +1/fixed_in_period, asset_weight applied</li> <li>Cache for 5 min (executives reload, no need to recompute)</li> <li>Add simple FP rule: if Trivy reports CVE for package version that doesn't match runtime fingerprint, mark fp_candidate=True</li> <li>Default findings view hides fp_candidate=True</li> </ul> | <ul style="list-style-type: none"> <li>Build /executive page: single-screen, designed for projection — text-2xl headings, large numbers, sparse layout</li> <li>Top: posture score (large circle, color-coded), week-over-week delta arrow</li> <li>Middle: 3 cards — Validated Critical · KEV in Wild · Reachable High Severity</li> <li>Bottom: AI exec paragraph (the same Claude exec_summary)</li> <li>'Present mode' button (lucide Maximize) → fullscreen, hides nav, larger text</li> <li>URL-shareable: /executive?as_of=2026-05-21 (asof date param)</li> <li>Build FP toggle in FindingsTable, 'Mark as FP' button in FindingDetailPanel</li> </ul> | <ul style="list-style-type: none"> <li>CISO view loads in &lt;500ms</li> <li>Present mode looks board-room-ready</li> <li>FP suppress works</li> </ul> |

| Day           | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                  | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                             | Done When                                                                                                                  |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
|               |                                                                                                                                                                                                                                                                                                                                                                                                                    | <ul style="list-style-type: none"> <li>• Tag: v0.5.0-month2-complete</li> </ul>                                                                                                                                                                                                                                         |                                                                                                                            |
| <b>Day 40</b> | <ul style="list-style-type: none"> <li>• Bug bash with Karthik: spend a full day clicking around, breaking things, fixing edge cases</li> <li>• Document known limitations in v1-LIMITATIONS.md (e.g. 'no cloud scanning yet', 'no SSO yet')</li> <li>• Update README with quickstart, screenshots, demo gif</li> <li>• Prepare 5-min demo script: login → new scan → findings → AI detail → Jira → PDF</li> </ul> | <ul style="list-style-type: none"> <li>• Same bug bash from the other side</li> <li>• Fix CSS regressions, broken empty states, weird hover states (Tailwind's group-hover, focus-visible)</li> <li>• Record screen capture of demo flow (use OBS or Loom)</li> <li>• Cut 30-sec teaser version for outreach</li> </ul> | <ul style="list-style-type: none"> <li>• Demo flow runs cleanly 3x in a row</li> <li>• Tag: v0.5.0-month2-final</li> </ul> |

## MONTH 3 — Production Deploy & First Customer · Weeks 9–12

End goal: Live on the internet, GitHub App live for PR-time scanning (FR-SCN-12), first paying customer onboarded.

### WEEK 9 — CI/CD, Docker prod, monitoring

| Day           | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                           | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Done When                                                                                                                              |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Day 41</b> | <ul style="list-style-type: none"> <li>Write Dockerfile for backend (multi-stage: python:3.12-slim base, copy app, install deps, non-root user)</li> <li>Write Dockerfile for worker (same base + add tool binaries: nuclei, trivy, gitleaks)</li> <li>Verify image sizes: aim for &lt;500MB worker (the scanners are heavy)</li> <li>Push to GitHub Container Registry: ghcr.io/your-org/platform-backend:0.5.0</li> </ul> | <ul style="list-style-type: none"> <li>Write Dockerfile for frontend (multi-stage: node:20-alpine build → standalone output → distroless or node:20-alpine runner)</li> <li>Use Next.js standalone output mode: in next.config.js set output: 'standalone' — produces minimal node_modules</li> <li>next build runs at image build time (env vars baked in via NEXT_PUBLIC_* — be careful with secrets)</li> <li>Push to ghcr.io/your-org/platform-frontend:0.5.0</li> </ul> | <ul style="list-style-type: none"> <li>All 3 images build + push</li> <li>Frontend image &lt;250MB with standalone mode</li> </ul>     |
| <b>Day 42</b> | <ul style="list-style-type: none"> <li>Write docker-compose.prod.yml: no source mounts, restart=unless-stopped, healthchecks on every service</li> <li>Add Caddy service: app.domain.com → frontend:3000, api.domain.com → backend:8000</li> <li>Test locally: docker compose -f docker-compose.prod.yml up → access via local hosts file</li> <li>Add .env.prod.example (different from dev defaults)</li> </ul>           | <ul style="list-style-type: none"> <li>Write .github/workflows/ci.yml: on push run ruff + pytest (backend); npm run lint + tsc --noEmit + vitest (frontend)</li> <li>Write .github/workflows/release.yml: on tag v* build + push images, version-tagged</li> <li>Add badge to README: build status</li> <li>Configure branch protection on main: PR required, CI must pass</li> </ul>                                                                                        | <ul style="list-style-type: none"> <li>Prod compose works locally</li> <li>CI runs green</li> <li>Branch protection on</li> </ul>      |
| <b>Day 43</b> | <ul style="list-style-type: none"> <li>Buy domain at Namecheap (~\$12/yr). Suggestion: short .com or .ai if available</li> <li>Set up Cloudflare in front (free tier): DNS, DDoS, basic WAF rules</li> <li>Set up Hetzner CX31 (4 vCPU, 8GB RAM, ~€17/mo) or DO/Vultr equivalent in EU/India</li> <li>Configure UFW firewall: allow 22, 80, 443 only; SSH key auth only, no passwords</li> </ul>                            | <ul style="list-style-type: none"> <li>SSH in, install Docker + Docker Compose plugin (1-liner from docker.com)</li> <li>Pull repo to /opt/platform; copy .env.prod with real secrets (generate JWT_SECRET via openssl rand -hex 32, AUTH_SECRET for Auth.js too)</li> <li>Set NEXTAUTH_URL=https://app.&lt;domain&gt; in .env.prod</li> <li>Run docker compose -f docker-compose.prod.yml up -d; verify all services healthy: docker compose ps</li> </ul>                  | <ul style="list-style-type: none"> <li>Domain resolves</li> <li>Server hardened</li> <li>Containers running on prod</li> </ul>         |
| <b>Day 44</b> | <ul style="list-style-type: none"> <li>Confirm SSL via Caddy (auto-issues Let's Encrypt cert on first request to https://app.&lt;domain&gt;)</li> <li>Verify: curl -I https://api.&lt;domain&gt;/health → 200 + valid cert</li> <li>Test login + scan end-to-end on production URL</li> <li>Tag: v1.0.0-rc1</li> </ul>                                                                                                      | <ul style="list-style-type: none"> <li>Run frontend through securityheaders.com — fix HSTS, CSP, X-Frame-Options gaps via Caddy headers</li> <li>Configure CSP carefully for Next.js: needs 'unsafe-inline' for inline styles unless you ship nonces (Next.js 15 supports CSP nonces via middleware)</li> <li>Run https://observatory.mozilla.org check — aim for A grade</li> <li>Test in incognito on phone — login + scan flow</li> <li>Tag: v1.0.0-rc1</li> </ul>        | <ul style="list-style-type: none"> <li>Green padlock everywhere</li> <li>Securityheaders A grade</li> <li>E2E works on prod</li> </ul> |

| Day    | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                        | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Done When                                                                                                                                 |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Day 45 | <ul style="list-style-type: none"> <li>Monitoring: Sentry (Python SDK) for backend errors. Free tier covers 5K events/mo</li> <li>Add SENTRY_DSN to .env, sentry_sdk.init in main.py</li> <li>DB backup cron on host: pg_dump → MinIO bucket 'backups' daily at 02:00, retain 30 days</li> <li>Test restore on local: docker run postgres, restore from a backup, verify data</li> </ul> | <ul style="list-style-type: none"> <li>Sentry for Next.js: npm i @sentry/nextjs, run npx @sentry/wizard@latest -i nextjs (auto-wires client + server + edge)</li> <li>Confirm sentry.client.config.ts, sentry.server.config.ts created; uncaught errors + Web Vitals captured</li> <li>UptimeRobot: monitor https://app.&lt;domain&gt;/ and https://api.&lt;domain&gt;/health every 5 min, SMS alert on 2-min downtime</li> <li>Test: throw new Error in a server action, confirm it lands in Sentry</li> </ul> | <ul style="list-style-type: none"> <li>Test errors in Sentry (both)</li> <li>Backup runs at 02:00</li> <li>UptimeRobot pinging</li> </ul> |

## WEEK 10 — Customer-ready polish + onboarding flow

| Day           | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                       | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Done When                                                                                                                             |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Day 46</b> | <ul style="list-style-type: none"> <li>Self-onboarding: POST /signup → create org + admin user (no manual provisioning)</li> <li>Email verification: send 6-digit code via Resend or SendGrid (free tier)</li> <li>Org slug = lowercase domain part of admin email (overrideable)</li> <li>Add billing-placeholder field on Org (we'll wire Stripe in v1.1)</li> </ul>                                                                                                  | <ul style="list-style-type: none"> <li>Build /signup page: shadcn Form with org name + admin email + password (zod schema enforces strength: min 12, mix of cases/digits)</li> <li>Show password strength visually with a small Progress bar that grows green as score increases</li> <li>Build /verify-email page: 6-digit OTP input using shadcn InputOTP component + resend button (60s cooldown)</li> <li>Build OnboardingWizard at /onboarding using shadcn Stepper pattern: Step 1 add first asset → Step 2 run first scan → Step 3 see findings</li> </ul> | <ul style="list-style-type: none"> <li>Signup → verify → onboard works</li> <li>Verified emails reach inbox</li> </ul>                |
| <b>Day 47</b> | <ul style="list-style-type: none"> <li>Pricing tier scaffolding (no Stripe yet, just enforcement): Free=10 scans/mo, Starter=100/mo, Pro=1000/mo, Enterprise=unlimited</li> <li>Check tier in middleware: 429 with 'Upgrade required' message + link</li> <li>Track scan_count per org per calendar month</li> <li>Reset on month rollover (Celery Beat task)</li> </ul>                                                                                                | <ul style="list-style-type: none"> <li>Build public /pricing page (no auth, lives outside (app) group) — 4 tier Cards, monthly + annual Switch, 'Start free' CTA</li> <li>Build /settings → Billing tab (admin only, role-gated): current tier · scans used / quota · 'Contact sales' button (Stripe v1.1)</li> <li>Add quota indicator pill in TopBar: '23/100 scans this month' (colored amber at 80%, red at 95%)</li> <li>Add upgrade Banner component (shadcn Alert variant) when &gt;80% quota</li> </ul>                                                   | <ul style="list-style-type: none"> <li>Free tier blocked at 10 scans</li> <li>Pricing page lives</li> <li>Quota visible</li> </ul>    |
| <b>Day 48</b> | <ul style="list-style-type: none"> <li>Write API docs auto-published at /docs (FastAPI built-in)</li> <li>Customize: add description per endpoint, examples, auth flow explanation</li> <li>Add public API key model: POST /api-keys → returns key; PATCH revokes</li> <li>Document rate limits, idempotency, webhooks signing</li> </ul>                                                                                                                               | <ul style="list-style-type: none"> <li>Build /docs in-app page with link to FastAPI /docs, plus inline 'Getting Started' guide with code samples</li> <li>Use shadcn Tabs with 'curl', 'Python', 'TypeScript' tabs; copy-to-clipboard button on each block (lucide Copy)</li> <li>Use a syntax highlighter: npm i shiki (lightweight, used by VS Code) — server-rendered code blocks</li> <li>Add Postman collection download link; public marketing pages: /, /pricing, /docs, /security (server components, fast TTFB)</li> </ul>                               | <ul style="list-style-type: none"> <li>API key works</li> <li>Docs followable by a dev with no help</li> </ul>                        |
| <b>Day 49</b> | <ul style="list-style-type: none"> <li>Write 'Security' page content (needed for SOC2 questions): encryption at rest, in transit, isolation model, no customer data in AI prompts beyond CVE metadata, deletion within 30d on request</li> <li>Write basic Privacy + ToS (use generator + manual review)</li> <li>Set up support email forwarded to a shared inbox</li> <li>Add 'Status' page link (use a free service like status.io or just a static page)</li> </ul> | <ul style="list-style-type: none"> <li>Build /security, /privacy, /terms pages as server components (mostly content, simple Tailwind prose typography via @tailwindcss/typography)</li> <li>Add cookie consent banner (no third-party cookies needed if you skip analytics — even better)</li> <li>Add support widget: small chat icon (lucide MessageCircle) bottom-right that opens mailto:support@&lt;domain&gt;</li> <li>Footer component with all the links, reused across (marketing) group</li> </ul>                                                      | <ul style="list-style-type: none"> <li>Public pages live</li> <li>Support email forwards</li> <li>Cookie banner shows once</li> </ul> |
| <b>Day 50</b> | <ul style="list-style-type: none"> <li>Write internal runbook: how to deploy, rollback, restore DB, rotate JWT secret, add new tool adapter</li> </ul>                                                                                                                                                                                                                                                                                                                  | <ul style="list-style-type: none"> <li>Write customer onboarding guide (2-page PDF): how to scan, how to read findings, how to create Jira tickets</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                     | <ul style="list-style-type: none"> <li>Runbook covers top 10 ops scenarios</li> </ul>                                                 |

| Day | Sathish — Backend / Security / AI                                                                                                                                                                                     | Karthik — Frontend / Deploy                                                                                                                                                                                                                             | Done When                                                                                         |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|     | <ul style="list-style-type: none"> <li>• Document 'oh shit' procedures: find a hammering user, block an abusive IP</li> <li>• Set up logging: structlog + JSON output → docker logs</li> <li>• Tag: v1.0.0</li> </ul> | <ul style="list-style-type: none"> <li>• Include screenshots of every key screen</li> <li>• Build /help in-app page with searchable FAQ (server-rendered, can be a JSON-driven list with client-side filter)</li> <li>• Tag frontend: v1.0.0</li> </ul> | <ul style="list-style-type: none"> <li>• Onboarding PDF ready</li> <li>• v1.0.0 tagged</li> </ul> |

## WEEK 11 — GitHub App for PR-time scanning (FR-SCN-12) + dogfooding + outreach

| Day    | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                                                                                                     | Done When                                                                                                                                                                    |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Day 51 | <ul style="list-style-type: none"> <li>• Dogfood: run platform against itself — point Nuclei at api.&lt;domain&gt;, Gitleaks at our own repo, Trivy at our own images</li> <li>• Fix anything it finds (also marketing material: 'We dogfood.')</li> <li>• Add daily auto-scan of self via Celery Beat → Slack on findings</li> <li>• Document 'How we secure our own platform' blog post</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <ul style="list-style-type: none"> <li>• Build /status page (use Better Stack free tier embed, or custom): API · App · Worker · DB · Scan engine</li> <li>• Show last 90 days uptime (server component fetching from UptimeRobot API at build time + ISR revalidate every 5 min)</li> <li>• Public access (link from footer)</li> <li>• Hook to UptimeRobot data</li> </ul>                                                                                     | <ul style="list-style-type: none"> <li>• Platform passes its own scan (zero critical)</li> <li>• Status page public</li> </ul>                                               |
| Day 52 | <ul style="list-style-type: none"> <li>• Build target customer list: 30 SMB/mid-market companies, or 30 cold prospects from LinkedIn</li> <li>• Write outreach template: short, specific, mention 1 thing you noticed about their public posture (or just be honest you're cold-emailing)</li> <li>• Send first 10 today</li> <li>• Track in a simple spreadsheet: sent · replied · demo · trial · paid</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <ul style="list-style-type: none"> <li>• Build /demo public page: embedded video walkthrough (5 min recorded earlier)</li> <li>• Add 'Book a demo' Calendly link (embedded iframe or just a link)</li> <li>• Build /case-study placeholder (will fill after first customer)</li> <li>• Add Plausible or Umami self-hosted analytics (no Google Analytics — privacy positioning) via a single script tag</li> </ul>                                              | <ul style="list-style-type: none"> <li>• 10 outreach emails sent</li> <li>• Demo page lives</li> </ul>                                                                       |
| Day 53 | <ul style="list-style-type: none"> <li>• FR-SCN-12 (Part 1) — GitHub App for PR-time scanning (DefectDojo structurally CANNOT do this)</li> <li>• Register GitHub App at github.com/settings/apps: permissions = repo contents (read), pull requests (write), checks (write)</li> <li>• Webhook URL: https://api.&lt;domain&gt;/github/webhook; subscribe to: pull_request (opened, synchronize)</li> <li>• Build app/github/webhook.py: verify HMAC signature, queue celery task pr_scan_task(repo, pr_number, sha)</li> <li>• pr_scan_task: clone repo at PR head SHA + base SHA, compute diff files, run scanners scoped to changed files</li> <li>• - Nuclei: skip (no web target on a PR)</li> <li>• - Trivy: scan only changed package files (requirements.txt, package.json, go.mod, Dockerfile)</li> <li>• - Gitleaks: scan only the diff (--log-opts='base..head')</li> </ul> | <ul style="list-style-type: none"> <li>• Build /settings → GitHub tab UI (extend Week 7 Day 33 GitHub config)</li> <li>• Show 'Install GitHub App' button → opens github.com/apps/{app-name}/installations/new</li> <li>• After install, callback to /github/callback stores installation_id linked to org</li> <li>• Show installed repos list with toggle (enabled/disabled per repo)</li> <li>• Send next 10 outreach emails (split with Sathish)</li> </ul> | <ul style="list-style-type: none"> <li>• GitHub App installs on test repo</li> <li>• PR webhook fires + queues scan task</li> <li>• 20 total outreach sent</li> </ul>        |
| Day 54 | <ul style="list-style-type: none"> <li>• FR-SCN-12 (Part 2) — Post results back to PR</li> <li>• Use GitHub Checks API: create check_run on PR head SHA with conclusion success/failure/neutral</li> <li>• Failure if any critical findings on changed files; neutral if medium/high; success if clean</li> <li>• Post PR comment with structured summary: 'Found 2 critical, 1 high in this PR — see details'</li> <li>• For each finding: file:line annotation via Checks API (inline in 'Files</li> </ul>                                                                                                                                                                                                                                                                                                                                                                           | <ul style="list-style-type: none"> <li>• Build /scans/pr/{repo}/{pr_number} page showing PR scan history (separate from on-demand /scans)</li> <li>• Demo prep: run through demo flow once per day with fresh test data so muscle memory is solid</li> <li>• Build a 'demo mode' flag (env or query param) that loads sample data — useful for screenshots and first-time exploration</li> <li>• Fix whatever demos surface from UX side</li> </ul>             | <ul style="list-style-type: none"> <li>• PR gets inline file:line annotations from scanner</li> <li>• PR comment summarizes findings</li> <li>• 30 outreach total</li> </ul> |

| Day    | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                 | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                            | Done When                                                                                                          |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
|        | <p>changed' tab)</p> <ul style="list-style-type: none"> <li>• Whatever the demos surface: missing feature, confusing UX, wrong default. Fix top 3 immediately</li> <li>• Send next 10 outreach emails</li> </ul>                                                                                                  |                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                    |
| Day 55 | <ul style="list-style-type: none"> <li>• Pricing pressure test: review tier limits against actual cost per scan (compute + AI tokens)</li> <li>• Adjust tier prices if needed</li> <li>• Add annual discount (20% off)</li> <li>• Document 'enterprise' tier: SSO, on-prem, custom SLAs, contact sales</li> </ul> | <ul style="list-style-type: none"> <li>• First demo this week if anyone said yes; if not, follow up with the 'no replies' batch</li> <li>• After each demo: 24h follow-up email with summary + next step + 14-day free trial activation link</li> <li>• Refine pitch based on what questions came up</li> <li>• Tag: v1.0.1-post-demo-fixes</li> </ul> | <ul style="list-style-type: none"> <li>• Pricing matches unit economics</li> <li>• 1+ demo done ideally</li> </ul> |

## WEEK 12 — First customer live + iteration

| Day           | Sathish — Backend / Security / AI                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Karthik — Frontend / Deploy                                                                                                                                                                                                                                                                                                                                                        | Done When                                                                                                                             |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Day 56</b> | <ul style="list-style-type: none"> <li>Onboard first paying customer (or first paid pilot): white-glove setup call, scan their assets together, walk through findings</li> <li>Document every friction point</li> <li>Fix critical ones same-day; queue medium ones</li> <li>Set up dedicated Slack channel for that customer's feedback</li> </ul>                                                                                                                                 | <ul style="list-style-type: none"> <li>Be on the white-glove call to observe UI confusion</li> <li>Take notes on what they look for and can't find</li> <li>Same-day fix top 2 UX issues</li> <li>Add 'product feedback' button in-app (lucide MessageSquare, opens dialog with textarea → POST /feedback)</li> </ul>                                                              | <ul style="list-style-type: none"> <li>First customer logged in + ran first scan</li> <li>Payment received (or LOI signed)</li> </ul> |
| <b>Day 57</b> | <ul style="list-style-type: none"> <li>Monitor customer usage: which scans, which findings did they click, did they create Jira tickets?</li> <li>Identify drop-off points — usually the gap between 'see finding' and 'do something about it'</li> <li>Add nudges: 'You have 3 critical findings older than 7 days — create Jira tickets?'</li> <li>Iterate on AI remediation prompts based on what their team finds useful vs not</li> </ul>                                      | <ul style="list-style-type: none"> <li>Build customer dashboard analytics (admin-facing, internal): how often they log in, scans/week, time-to-Jira</li> <li>These become product KPIs</li> <li>Send proactive 'health check' email to customer at end of week 1</li> <li>Improve onboarding wizard based on what tripped them up</li> </ul>                                       | <ul style="list-style-type: none"> <li>Customer KPIs visible to us</li> <li>Customer engaged in week 1</li> </ul>                     |
| <b>Day 58</b> | <ul style="list-style-type: none"> <li>Cost analysis: real Anthropic spend per scan, real server cost. Confirm unit economics (target: &lt;30% COGS at Starter tier)</li> <li>If unit economics off: add prompt caching for common CVE contexts, batch findings tighter, drop max_tokens if room</li> <li>Run a chaos test: kill worker mid-scan, verify scan retries, no orphaned data</li> <li>Document 12-week retrospective: what shipped, what slipped, what's next</li> </ul> | <ul style="list-style-type: none"> <li>Marketing: write blog post #1 — 'Why we built X' (the story)</li> <li>Marketing: blog post #2 — 'Our first customer found 47 CVEs in 12 minutes' (case study, anonymized if needed)</li> <li>Submit to /r/cybersecurity, /r/devops, HackerNews 'Show HN', LinkedIn</li> <li>Iterate landing page based on first week's analytics</li> </ul> | <ul style="list-style-type: none"> <li>Unit economics positive at Starter</li> <li>2 blog posts live</li> </ul>                       |
| <b>Day 59</b> | <ul style="list-style-type: none"> <li>Plan v1.1 backlog (the deferred features): Prowler cloud · Checkov IaC · Nmap · OWASP heatmap · MITRE clusters · compliance reports · recurring scans · SSO</li> <li>Rank by what customers asked for vs what we think is impressive</li> <li>Customer pull &gt; engineering push — almost always</li> <li>Add 'feature voting' on a public roadmap (Canny or GitHub Issues with labels)</li> </ul>                                          | <ul style="list-style-type: none"> <li>Build /roadmap public page: shows 'shipped · in progress · next · later' columns (4-column grid, shadcn Cards)</li> <li>Drives customer trust + helps prioritize</li> <li>Add 'request a feature' form (just POSTs to /feedback or emails support)</li> <li>Tag: v1.0.2-with-feedback</li> </ul>                                            | <ul style="list-style-type: none"> <li>Roadmap public</li> <li>v1.1 backlog has 10–15 items prioritized</li> </ul>                    |
| <b>Day 60</b> | <ul style="list-style-type: none"> <li>Final sprint retrospective with Karthik: what worked, what didn't, what's next</li> <li>Decide team structure: keep this split, or both work on whatever customers pull?</li> <li>Set targets for Month 4: customers, ARR, scan volume, NPS</li> <li>Celebrate. You shipped a real product in 12 weeks.</li> </ul>                                                                                                                           | <ul style="list-style-type: none"> <li>Same retrospective</li> <li>Update both resumes with the project (major portfolio item)</li> <li>Plan Month 4 split (likely: Karthik on Stripe + SSO, Sathish on Prowler + compliance — the v1.1 sequence)</li> <li>Tag: v1.0.0-shipped</li> </ul>                                                                                          | <ul style="list-style-type: none"> <li>Retrospective documented</li> <li>Month 4 plan set</li> <li>Resumes updated</li> </ul>         |



## 12-WEEK SUMMARY

| Month          | Phase                    | Week Range | Milestone                                                                                                                                                                                                                                                  |
|----------------|--------------------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Month 1</b> | Foundation & First Scans | Weeks 1–4  | Multi-tenant API + 3 scanners (Nuclei, Trivy, Gitleaks) producing findings in DB. Login via Auth.js → FastAPI. Exploit validation (FR-SCN-13) on critical findings. Empty-but-functional Next.js UI with shadcn components.                                |
| <b>Month 2</b> | Intelligence & AI        | Weeks 5–8  | Every finding enriched with CVSS + EPSS + KEV + Claude summary. Source-call reachability (FR-AI-08), agentic chat (FR-AI-10), AI-generated fix PRs (FR-OUT-04), single-screen CISO view (FR-UI-08), Slack threads per finding. Jira + PDF outputs working. |
| <b>Month 3</b> | Production & Customer    | Weeks 9–12 | Live on production with monitoring, billing tiers, signup flow. GitHub App for PR-time scanning (FR-SCN-12) goes live. First paying customer onboarded. v1.1 roadmap defined.                                                                              |

### Rules for staying on schedule

- Never start a phase until previous phase's tag is pushed. Phase 3 adapters WILL fail if Phase 2 backend is incomplete.
- Demo at end of each week to each other. If you can't demo it, it's not done.
- Cut scope before cutting quality. Defer scanners, don't ship a broken UI.
- One bug bash day per week (Friday). Don't accumulate tech debt past 5 days.
- Customer pull > engineering push from Week 10 onwards. Build what they ask for, not what we think is cool.
- Both review each other's code via PR. Solo merges to main are forbidden once CI is set up.

### Tech-stack split summary

Sathish: Python 3.12 · FastAPI · Pydantic v2 · SQLAlchemy · Alembic · Celery · Redis · PostgreSQL · MinIO · Nuclei · Trivy · Gitleaks · NVD/KEV/EPSS API clients · Anthropic Python SDK · Jira + Slack + ReportLab

Karthik: Next.js 15 (App Router) · React 18 · TypeScript · Tailwind CSS · shadcn/ui · Auth.js v5 · TanStack Query · TanStack Table · react-hook-form · zod · Recharts · sonner · lucide-react · next-themes · Inter font · react-dropzone · Docker · Docker Compose · GitHub Actions · Caddy · Sentry · UptimeRobot · Cloudflare DNS

Both: git · GitHub · GitHub Container Registry · Postman/Bruno · Sentry · UptimeRobot · runbook · customer onboarding

**END OF PLAN**